yAudit

June 2026

Prepared for
Yearn

Audited by
panda
adriro

Yearn Tokenized Strategy v3.1.0

Smart Contract Security Assessment

Contents

1 Review Summary

1.1 Protocol Overview

Yearn Tokenized Strategy provides a reusable framework for deploying single-strategy ERC-4626 vaults that are compatible with the Yearn V3 Vaults infrastructure.

1.2 Audit Scope

This audit covers 3 smart contracts totaling approximately 1379 lines of code across 4 days of review.

```
src
├── BaseStrategy.sol
├── TokenizedStrategy.sol
└── libraries
    └── TokenizedStrategyLib.sol
```

1.3 Risk Assessment Framework

1.3.1 Severity Classification

Severity	Description	Potential Impact
Critical	Immediate threat to user funds or protocol integrity	Direct loss of funds, protocol compromise
High	Significant security risk requiring urgent attention	Potential fund loss, major functionality disruption
Medium	Important issue that should be addressed	Limited fund risk, functionality concerns
Low	Minor issue with minimal impact	Best practice violations, minor inefficiencies
Undetermined	Findings whose impact could not be fully assessed within the time constraints of the engagement. These issues may range from low to critical severity, and although their exact consequences remain uncertain, they present a sufficient potential risk to warrant attention and remediation.	Varies based on actual severity
Gas	Findings that can improve the gas efficiency of the contracts.	Increased transaction costs
Informational	Code quality and best practice recommendations	Reduced maintainability and readability

Table 1: severity classification

1.4 Key Findings

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	1
Medium	1
Low	2
Informational	3

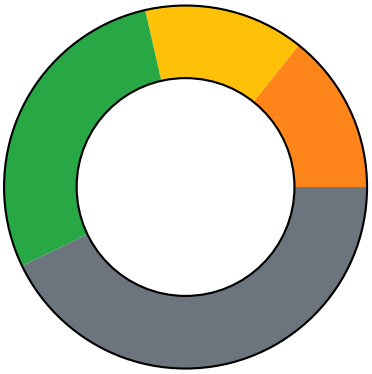

Figure 1: Distribution of security findings by impact level

1.5 Overall Assessment

The review identified one high-severity share-price inflation issue and several accounting and integration risks requiring remediation.

2 Audit Overview

2.1 Project Information

Protocol Name: Yearn
Repository: https://github.com/yearn/tokenized-strategy
Commit Hash: 5de1a435899a83bec1502da513c908c75abc95b2
Final Commit Hash:
Commit URL: https://github.com/yearn/tokenized-strategy/commit/5de1a435899a83bec1502da513c908c75abc95b2

2.2 Audit Timeline

The audit was conducted from June 9 to 12, 2026.

2.3 Audit Resources

- Code repositories and documentation

Category	Mark	Description
Access Control	Average	Role boundaries are generally explicit, but the receiver-based deposit-limit hook can be mistaken for caller authorization, allowing untrusted callers to trigger deposit-time deployment through an approved receiver.
Mathematics	Low	Share conversion and accrual accounting contain material edge cases. A first depositor can manipulate rounding through a direct donation, and loss checkpoints can cause principal recovery to be charged performance fees.
Complexity	Average	Constant accrual, simulated totals, locked-share loss handling, and strategy hooks create subtle accounting-order dependencies. The interaction between tend, emergency withdrawal, and later accrual is difficult for integrators to reason about safely.
Libraries	Good	The reviewed framework is self-contained and no material issue was identified in its use of libraries or external dependencies.
Decentralization	Good	The findings did not identify a material centralization or governance weakness; privileged strategy operations remain within the framework's documented management model.
Documentation	Average	Documentation is extensive, but some guidance no longer matches constant-accrual behavior. In particular, tend, emergency withdrawal, and deposit-limit comments can lead strategy authors to assume accounting or authorization guarantees that are not enforced.
Testing and verification	Average	The existing tests did not cover several adversarial accounting paths, including initial-deposit donation inflation, loss-recovery fee charging, and hook-side state changes under constant accrual.

Table 2: Code Evaluation Matrix

2.4 Critical Findings

None.

2.5 High Findings

2.5.1 Strategy is subject to multiple block infation attacks

The first depositor can inflate the share price with a direct donation and steal part of the next depositor's principal through rounding.

Technical Details

When the strategy has no shares, `_convertToSharesFromTotals()` mints shares at a 1:1 ratio. An attacker can therefore deposit one asset unit and become the sole shareholder.
The attacker can then transfer assets directly to the strategy before a victim's `deposit()` transaction. On the victim's deposit, `_accrue()` recognizes the donation as profit, but the dead-share protection does not execute because it requires both `oldTotalAssets == 0` and raw `S.totalSupply == 0`. The attacker's initial one-unit deposit makes both conditions false.
A focused Foundry test reproduced the following sequence:

1. The attacker deposits `1` wei and receives `1` it can be executed when a strategy is deployed.
2. The attacker donates `100e18` assets directly to the strategy.
3. The victim deposits `200e18` assets and receives only `1` share.
4. The attacker redeems their share for `150e18` assets.
5. The victim can redeem only `150e18 + 1` assets.

The attacker contributed `100e18 + 1` and earned almost `50e18`, while the victim lost almost `50e18`. The dead-share balance remains zero throughout the attack. The attacker generally needs to fund the donation across the victim's separate transaction, although the capital can be recovered immediately in a backrun after the victim deposits.

Impact

High.

Recommendation

Require a sufficiently large minimum initial deposit and permanently lock or burn a fixed portion of the initial shares as minimum liquidity.

Developer Response

Added a min supply value of 1e3 instead of just checking !=0 for the total supply that makes it prohibitive to do this style of attack. Any profit realized when totalSupply < 1e3 will be attributed to the burn address.
e9ee3e83

2.6 Medium Findings

2.6.1 Constant accrual conflicts with tend() and emergencyWithdraw() semantics

`tend()` and `emergencyWithdraw()` still behave and are documented as report-oriented hooks, but constant accrual makes their accounting effects visible through `_simulatedTotals()` and the next `_accrue()` call. This creates a semantic conflict for strategies that use these hooks to harvest, rebalance, unwind positions, or otherwise mutate economic state.

Technical Details

`TokenizedStrategy` checkpoints live accounting with `_accrue()` before user-facing ERC4626 flows and some management updates, but `tend()` directly calls `tendThis()` and `emergencyWithdraw()` directly calls `shutdownWithdraw()` without first checkpointing the current strategy state. Both callbacks can execute strategy-defined mutable logic.
This means pre-hook profit or loss can be transformed by `_tend()` or `_emergencyWithdraw()` before it is accounted. For example, live profit that should have paid fees can be partially consumed by slippage during a rebalance, so the later `_accrue()` only sees the net result. Similarly, a loss can be netted against unreported profit instead of being realized in the intended order and offset against locked shares where applicable.
The surrounding `BaseStrategy` documentation still states that `_tend()` has no PPS effect until `report()` and that `_emergencyWithdraw()` will not realize profit or loss without a later `report()`. That invariant no longer holds with constant accrual. Post-hook changes to `strategyTotalAssets()` can affect view pricing through `_simulatedTotals()` and can be realized by the next mutable `_accrue()` call triggered by deposit, mint, withdraw, redeem, fee configuration, or report.

Impact

Medium. Impact mainly depends on the concrete strategist implementation. If `_tend()` and `_emergencyWithdraw()` are strictly value-neutral, the issue is primarily a documentation and integration hazard. If they harvest, swap, compound, rebalance, unwind, or incur slippage or penalties, accounting can be checkpointed in an unexpected order, fees can be undercharged, losses can be netted incorrectly, and locked-share loss handling can be bypassed for the pre-hook delta.

Recommendation

Clearly document that `_tend()` and `_emergencyWithdraw()` are no longer report-only accounting boundaries under constant accrual. Strategy authors should be aware that mutations made by these hooks can affect simulated views and can be realized by the next `_accrue()` call. Consider calling `_accrue()` at the start of `tend()` before invoking the strategy hook, so pre-hook profit and loss are checkpointed before mutable interaction. Also document the same-block latch behavior so strategists know whether hook-side changes are intended to be reflected immediately, in the next block, or only after an explicit report.

Developer Response

Intentional. Documentation updated to reflect the current state in PR#120.

Emergency withdrawal is meant to be as simple as possible since it is being called in unstable times. Adding more hooks and internal logic that can potentially revert or cause problems would defeat that purpose.
Tend should not accrue profits and lock the blocks accounting before updating its state. In case of losses realised on the trend, that would mean causing incorrect fees to be charged. In case of profits realised, there is no reason not to allow the next touch point to be based on that PPS.

2.7 Low Findings

2.7.1 Receiver-based deposit limit hook does not authenticate deposit callers

Technical Details

The `TokenizedStrategy` implementation checks deposit eligibility against the share `receiver` rather than the actual `msg.sender` who provides the assets. When `deposit()` or `mint()` is called, `_maxDeposit()` and `_maxMint()` invoke `availableDepositLimit(receiver)`. This passes the `receiver` parameter to the hook without any check against `msg.sender`.
The deposit flow then transfers assets from `msg.sender` via `safeTransferFrom()` and immediately calls `deployFunds(asset.balanceOf(address(this)))`. Because `deployFunds()` deploys the full loose balance, an attacker can supply a minimal deposit amount to any whitelisted receiver and still trigger deployment of the entire idle balance held by the strategy.
`BaseStrategy` documentation describes `availableDepositLimit()` as applying to ”the address that is depositing into the strategy” and presents a whitelist example keyed by `_owner`. The same file further states that `_deployFunds()` is ”entirely permissionless and thus can be sandwiched or otherwise manipulated” unless a whitelist is implemented. A strategist following this guidance would override `availableDepositLimit()` expecting caller-level protection, but the framework only validates the receiver of shares.
This mismatch means that a strategy intending to restrict who can invoke deposit-time deployment will fail to enforce that restriction. An untrusted caller can bypass the intended whitelist by specifying an allowed receiver, effectively gaining permissionless access to the `_deployFunds()` callback.

Impact

Low. An attacker can force deployment of a strategy’s idle assets at arbitrary timing by depositing a minimal amount to any whitelisted receiver.

Recommendation

Do not rely on `availableDepositLimit()` for caller authentication. Introduce a separate authorization hook that receives `msg.sender` and validates the caller before `_deposit()` executes, or require strategies needing permissioned deposits to override `deposit()` and `mint()` with an explicit `msg.sender` check. At minimum, update the documentation to clarify that `availableDepositLimit()` is receiver-based and remove guidance implying it provides caller-level protection. For example, add a comment noting that the hook controls share issuance but does not restrict who can invoke the deposit path, and recommend implementing caller validation separately if deposit-time callbacks must be permissioned.

Developer Response

Known. The deposit limit functions are view functions and only pass in the receiver parameter. Strategists are expected to implement accordingly and there are periphery contracts that allow for msg sender contract for those that need it.
Comment in BaseStrategy updated PR#120.

2.7.2 Permissionless loss checkpoints cause performance fees to be charged on principal recovery

Technical Details

Every public ERC-4626 write first calls `_accrue()`. When the strategy has lost value, `_accrue()` records the lower value directly in `S.lastTotalAssets`. If the strategy later recovers, the difference from that reduced checkpoint is treated as fresh profit and passed to `_chargeFees()`, even when the strategy has only returned to its value before the loss.
A user action can crystallize both checkpoints in different timestamps:

1. An existing holder deposits 1,000 assets.
2. The strategy temporarily loses 100 assets and a user deposits, updating `lastTotalAssets` to the reduced value.
3. The strategy recovers the same 100 assets and a later user action triggers accounting again.
4. `_accrue()` treats the recovery as 100 assets of profit and mints performance-fee shares.

Impact

Low.

Recommendation

A loss should increase the amount that must be recovered before future gains become fee-eligible.

Developer Response

Known/Accepted

2.8 Gas Savings Findings

2.8.1 report() recomputes old total assets after accrual

`report()` calls `_totalAssets(S)` to fetch the old accounting baseline after `_accrue(S)` has already synchronized `S.lastTotalAssets`. Reading `S.lastTotalAssets` directly would avoid unnecessary internal view logic.

Technical Details

At the start of `report()`, `_accrue(S)` updates `S.lastTotalAssets` to the current read-only `strategyTotalAssets()` value and latches `S.lastAccrual`. After `harvestAndReport()` returns the post-harvest amount, `report()` computes `oldTotalAssets` with `_totalAssets(S)`. `_totalAssets(S)` routes through `_simulatedTotals(S)`. Since `report()` is non-reentrant and executes with `S.entered == ENTERED`, `_simulatedTotals(S)` returns `S.lastTotalAssets` instead of querying fresh strategy assets. Therefore, the call does extra supply calculation and branching only to recover the already-available storage value.

Impact

Gas Savings.

Recommendation

Use `S.lastTotalAssets` directly for `oldTotalAssets` in `report()`.

Developer Response

Changed in PR#120.

2.9 Informational Findings

2.9.1 TokenizedStrategyLib duplicates the storage layout

`TokenizedStrategyLib` duplicates `TokenizedStrategy`'s storage slot and `StrategyData` layout. This currently matches the canonical layout, but future field additions or reordering must be manually synchronized across both definitions.

Technical Details

Both `TokenizedStrategy` and `TokenizedStrategyLib` define the same `StrategyData` struct used as the contract storage layout.
Any future update to `TokenizedStrategy.StrategyData` that is not reflected in `TokenizedStrategyLib.StrategyData` can make the library read the wrong slots or misinterpret packed fields. The risk is maintainability related, but the affected fields include critical accounting and access-control state.

Impact

Informational.

Recommendation

Move the shared storage slot constant, `StrategyData` struct, and storage accessor into a single internal storage library that both `TokenizedStrategy` and `TokenizedStrategyLib` import. Optionally migrate the namespace declaration to the EIP-7201 namespaced storage pattern by deriving the root slot with the EIP-7201 formula. This would make the custom storage namespace easier for tooling and reviewers to recognize.

Developer Response

Intentional.

2.9.2 profitMaxUnlockTime() normalization is duplicated

`TokenizedStrategyLib.profitMaxUnlockTime()` repeats the sentinel conversion from `TokenizedStrategy.profitMaxUnlockTime()`. This does not currently change behavior, but it creates a second implementation of the same getter rule.

Technical Details

The public getter reads the packed `profitMaxUnlockTime` value and returns `type(uint256).max` when the stored value equals `type(uint32).max`. The library helper repeats the same logic against `strategyStorage().profitMaxUnlockTime`, while nearby library getters such as `symbol()`, `apiVersion()`, `MAX_FEE()`, and `FACTORY()` already self-call through `ITokenizedStrategy(address(this))`.
If the sentinel encoding or normalization changes later, both implementations must be updated together. A mismatch would expose different values depending on whether a strategy uses the external getter or the library helper.

Impact

Informational.

Recommendation

Have `TokenizedStrategyLib.profitMaxUnlockTime()` call `ITokenizedStrategy(address(this)).profitMaxUnlockTime()`.

Developer Response

Intentional.

2.9.3 totalSupply() uses non-simulated supply while accounting views use simulated totals

`totalSupply()` returns the realized supply state, while `totalAssets()` and conversion views use `_simulatedTotals()` to reflect pending constant accrual effects. This can make the live

view surface appear inconsistent when pending profit fees or loss burns are only simulated and have not been realized by a mutable `_accrue()` call.

Technical Details

`totalSupply()` returns `_totalSupply()` directly. This only accounts for the stored `S.totalSupply` minus unlocked strategy-held shares. It does not include fee shares that would be minted for pending constant-accrual profit, and it does not reflect the locked-share supply reduction that `_simulatedTotals()` applies for pending losses.
By contrast, `totalAssets()` calls `_simulatedTotals()` and returns the simulated asset value. `convertToAssets()`, `convertToShares()`, and preview functions also route through `_simulatedTotals()`, so their pricing can reflect the pending constant-accrual state before it is realized by a state-changing `_accrue()` call.
This may be intentional because fee shares and loss burns are not effectively realized in a non-mutable simulation. However, integrators that compare `totalSupply()` against `totalAssets()` or conversion outputs may observe different live accounting assumptions across these views.

Impact

Informational.

Recommendation

Document that `totalSupply()` exposes realized ERC20 supply accounting, while `totalAssets()`, conversions, and previews may expose simulated constant-accrual accounting. The documentation should make clear that pending fee-share minting and loss-related supply effects are reflected in conversion math before they are reflected in `totalSupply()`.

Developer Response

Intentional. Documentation added in PR#120.

Yearn Tokenized Strategy v3.1.0

Completed 2026-06-12